SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

In Malaysia, the challenge in charitable giving is not a lack of generosity, but a lack of coordination. Donors often do not know which organizations require help, while organizations struggle to communicate their needs in real time. During disasters, this gap becomes even more critical, leading to delayed or misdirected aid. DonateAI is a web-based platform designed to bridge this gap by connecting donors with verified organizations through an intelligent and structured system. It supports both everyday donation activities and disaster response scenarios. In normal situations, the platform enables donors to browse live needs, submit item donations with AI assistance, and track delivery progress. In crisis situations, the system shifts into a disaster-response mode, where it collects external signals, coordinates with shelters, and prioritizes urgent needs. By combining automation, AI support, and structured workflows, DonateAI ensures that donations are matched efficiently and delivered transparently.

1.1. Purpose:

This System Analysis Document (SAD) highlights the technical scope and design-related decisions behind the development of DonateAI.

The key elements covered include:

1. ***Architecture**:*
    a. *DonateAI is a cloud-ready web platform built using Next.js with a modular monolithic architecture. The system integrates multiple user portals (public, donor, receiver, and admin) with backend API route handlers. Supabase is used for database management, authentication, and role-based access control. AI services, including GLM, are integrated as a service layer to support intelligent donation matching, image verification, and data extraction.*

2. ***Data Flows:***
    a. *The system manages two major data flows. The first is the donation flow, where a donor submits item information through an AI-assisted interface, the system processes and matches it with live organizational needs, stores it in the database, and returns confirmation with tracking details. The second is the disaster-response flow, where external signals (such as news and alerts) are ingested, processed into structured events, and converted into live needs that donors can respond to.*

3. ***Model Process:***
    a. *The system supports an end-to-end workflow consisting of AI-assisted donation intake, allocation planning, and delivery tracking. It also includes a disaster-response workflow where signals are detected, shelters are contacted, responses are processed into structured needs, and donations are routed accordingly.*

© UMHackathon 2026

4. ***Role of Reference:***
    a. *This document serves as a reference for developers, testers, and stakeholders to understand the system architecture, AI integration, data flow, and functional scope of DonateAI.*

1.2. Background:

Malaysia is one of the countries frequently affected by natural disasters such as floods, which create urgent demand for coordinated relief efforts. However, the current donation ecosystem is fragmented, relying heavily on social media, messaging platforms, and informal communication channels.

As a result, donors face difficulty in identifying verified organizations that genuinely require help. At the same time, shelters and organizations often communicate their needs through unstructured messages, making it difficult to convert these requests into actionable data.

Even in normal situations, individuals who want to donate physical items lack a centralized and reliable platform to view real-time needs and ensure that their donations reach the correct recipients.

DonateAI is developed to solve these issues by providing a structured, AI-assisted platform that connects donors with organizations while ensuring efficient coordination and transparency.

1. *Previous Version:*
    a. *The earlier version of DonateAI was a basic donation matching web application with limited functionality.*
        - *Donors browsed a static list of organizations and predefined needs.*
        - *The system did not use real-time data for matching donations.*
        - *Donations were submitted through basic forms without verification or tracking.*
        - *There was no receiver portal for organizations to manage their needs.*
        - *There was no admin panel or disaster-response functionality.*
        - *The system was built as a frontend-only application without a backend service layer.*

    This version lacked scalability, automation, and coordination features required for real-world usage.

2. *Changes in Major Architectural Components*

    The current version introduces major improvements across several components:

    - *The system has been upgraded to a **full-stack web platform** using Next.js with server-side API routes.*
    - *A **backend service layer** is introduced to handle authentication, data processing, and business logic.*

© UMHackathon 2026

- ***Supabase integration*** *enables real-time database operations, authentication, and role-based security.*
- *AI capabilities using* ***GLM*** *are added for intelligent donation processing.*
- *A* ***multi-role system*** *is implemented, supporting public users, donors, receivers, and administrators.*
- *A* ***disaster-response module*** *is introduced, including signal ingestion, event activation, and shelter coordination.*
- *An* ***admin panel*** *is developed to manage disaster operations, communication, routing, and reporting.*
- *A* ***logistics system*** *is implemented for route planning, delivery tracking, and proof verification.*

3. *New capabilities that are only introduced in this*
   - *Dual-mode system: Supports both normal donation operations and disaster-response scenarios*
   - *AI-assisted donation workflow: Includes chatbot-based intake, image verification, and allocation planning*
   - *Shelter communication system: Uses WhatsApp integration to collect real-time needs from shelters*
   - *Automated need extraction: Converts unstructured messages into structured data using AI*
   - *Route planning and logistics management: Generates delivery routes and tracks progress*
   - *Equity monitoring: Identifies underserved shelters and ensures fair distribution of resources*
   - *Administrative control panel: Provides tools for managing events, communications, and system operations*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Individual Donors	*Initiates donation via conversational chat, describes items in natural language*	*Easy, guided donation process*
Organizations in need	*Publishes needs catalog, receives matched donations*	*Accurate and relevant donations*

© UMHackathon 2026

Corporate Sponsors	*Submits bulk asset donations via asynchronous form*	*Audit trail, ESG compliance reporting, wallet-based logistics funding*
Logistics Drivers	*Deliver items*	*Clear routing and task info*
Admin	*Manage disaster events, communications, and logistics*	*Full control over system operations and real-time monitoring*
Development Team	*Builds and maintains the platform*	*Clear API contracts, modular codebase, documented LLM prompt schemas*
QA Team	*Validates system behavior across all user roles*	*Defined request/response formats, LLM fallback coverage, edge case specs*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Browser-based platform for Public, Donor, Receiver, Admin)*
Architecture	*Cloud-based Modular Monolithic Architecture (Next.js with API Routes + Supabase Backend)*

*DonateAI is structured as a **client-server system** consisting of multiple user interfaces that interact with a centralized backend. All users (donors, receivers, admin) access the system through a web interface.*

The backend handles core operations such as donation processing, disaster coordination, AI interaction, and logistics management. Supabase is used for database storage and authentication, while external APIs provide AI processing, communication, and routing functionalities.

2.1.2 LLM as Service Layer

The system integrates AI (GLM and Gemini Vision) as a service layer to enhance decision-making and automation.

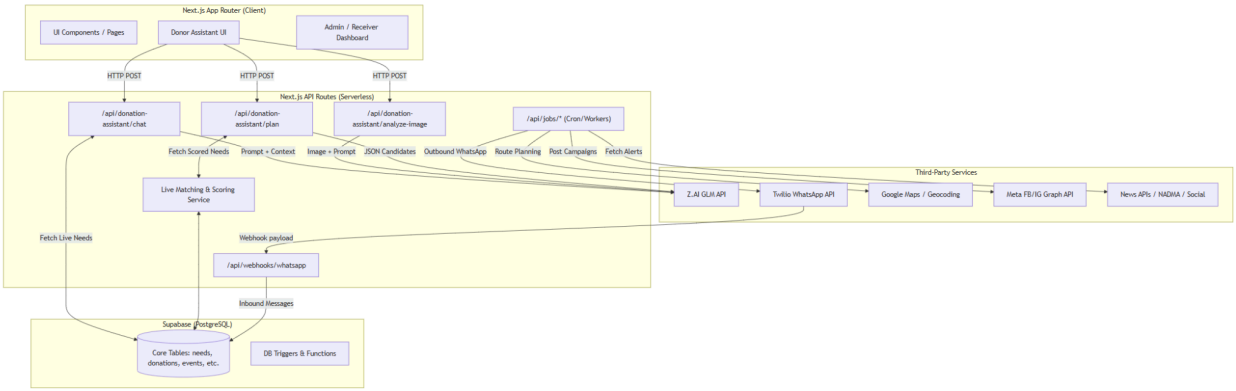
AI is used for:

- understanding donor input
- validating item images
- extracting structured needs from unstructured messages

The backend constructs prompts using system context and sends them to the AI services. The responses are then parsed and used in system workflows such as matching and allocation.

2.1.3 Dependency Diagram

© UMHackathon 2026



a. *How the prompts are being constructed and sent to the GLM API*

The system uses a highly structured, state-machine approach to build prompts dynamically based on the current step in the user journey. The logic is orchestrated in the backend and is divided into specific sections:

- *Intake Policy: Defines the strict rules for the GLM. It instructs the AI to only ask for 4 specific fields (Category, Quantity, Condition, Location), enforce the one-question-per-turn rule, and adhere to strict delivery transport rules (no paid couriers).*
- *Workflow Orchestration: Tells the AI its exact role depending on the API route being called (e.g., chat vs. vision vs. plan). It injects the current UI state (e.g., greeting, details, allocation) so the AI aligns its tone.*
- *Response Contracts: Applies strict output constraints based on the exact conversational stage. For instance, if the stage is details, it enforces a warm opening, asks only one missing intake field, and keeps the response under 90 words.*

b. *What goes into the context window?*

When a request is sent to the GLM API, the context window is populated with:

- *The System Prompts (Policy + Orchestration + Contracts).*
- *The Donor Transcript (Recent chat history).*
- *The Detected Item Category & Condition.*
- *Live Needs Data (Grounding): Instead of giving the LLM free rein, the system fetches live needs from Supabase. These records are passed as a structured JSON block for the LLM to reason over.*
- *Logistics Preferences (e.g., drop-off vs. platform-delivery).*

c. *How is your system receiving, parsing the responses and passing to the next system component?*

Because the system relies heavily on structured output, it handles AI responses differently depending on the stage:

- *Chat Stage: The GLM returns conversational plain text guidance, which is passed directly back to the frontend to render in the chat UI.*
- *Planning Stage:*
- *The GLM is strictly instructed (via prompt) to output JSON only, outlining how a donation should be split (e.g., 60% to Need A, 40% to Need B) along with reasoning bullets.*
- *Parsing: The Next.js API route parses this JSON payload.*
- *Validation (Crucial Step): The API validates the GLM's output. It checks if the needId returned by the AI actually exists in the live candidate set previously provided.*
- *Normalization: The API normalizes the percentage allocations to ensure they exactly sum to 100%.*
- *Handoff: The validated data is reshaped into the UI's expected PlanReceiver[] interface and sent back to the Next.js Client to render the allocation cards.*

d. *Where the limitation of token are enforced or input are chunked before it actually reaches to GLM*

To prevent overwhelming the GLM context window (token overflow) and to reduce hallucination, the system does not dump the entire database of disaster needs into the prompt.

Enforcement via Heuristic Filtering: Before the GLM API is called, the backend runs a deterministic scoring function (scoreNeedForDonation).

- *The backend reads the donor's text and extracts keywords.*
- *It scores active database records based on urgency, remaining quantity, disaster severity, and category boosts (e.g., mapping "rice" to "food").*
- *The limit: The query fetches a maximum of 80 needs, scores them, and only extracts the top-scoring candidates.*
- *Only these highly relevant candidates (getLiveNeedMatchCandidates) are converted into a lightweight JSON array and injected into the GLM prompt.*

This serves as a semantic "chunking" mechanism which uses deterministic code to narrow the search space before invoking expensive LLM reasoning.

e. *Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]*

The system uses Next.js 15 (App Router), where the frontend (Client Components) communicates with the backend (Serverless API Routes). The backend then orchestrates calls to the database and external AI/third-party services.

Donor AI Assistant Flow:

- *Next.js Client (Donor UI) calls /api/donation-assistant/chat to process the user's conversational input and request matching suggestions.*

© UMHackathon 2026

- *Backend (Chat Route) calls Supabase API to query active needs, organizations, and disaster_events for heuristic scoring.*
- *Backend (Chat Route) calls Z.AI GLM API to generate conversational guidance based on the prompt, live needs context, and transcript.*
- *Next.js Client (Donor UI) calls /api/donation-assistant/analyze-image when a donor uploads a photo.*
- *Backend (Vision Route) calls Z.AI GLM API (vision model) to screen the item's condition and validate its category.*
- *Next.js Client (Donor UI) calls /api/donation-assistant/plan after photo validation to finalize the allocation.*
- *Backend (Plan Route) calls Z.AI GLM API with pre-scored database candidate needs to generate a JSON-based percentage allocation.*

Disaster Automation & Integrations (Job Runners):

- *External Cron/Trigger calls /api/jobs/disaster-signal-ingest to initiate the disaster detection cycle.*
- *Backend (Ingest Job) calls External News APIs / NADMA / Social Feeds to fetch raw incident data.*
- *Backend (Ingest Job) calls Supabase API to persist structured incident_signals and auto-create pending disaster_events.*
- *External Cron/Trigger calls /api/jobs/route-planning to orchestrate delivery logistics.*
- *Backend (Route Job) calls Google Maps API (Distance Matrix/Geocoding) to calculate map travel estimates and nearest collection points.*
- *Backend (Route Job) calls Supabase API to persist route_plans, route_stops, and update route_allocations.*

External Communications (WhatsApp):

- *Backend (Outreach Job) calls Twilio API (WhatsApp) to send automated outreach messages to registered shelters/organizations.*
- *Twilio Webhook calls /api/webhooks/whatsapp with the inbound reply payload from a shelter.*
- *Backend (Webhook Route) calls Z.AI GLM API to extract structured disaster needs from the messy shelter reply text.*
- *Backend (Webhook Route) calls Supabase API to persist the new needs and log the communications record.*

f. Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]

1. Frontend (Next.js Client Components)

© UMHackathon 2026

- *Role: Handles user intent (donors dropping items, admins viewing crisis dashboards, receivers managing capacity). It manages local React state and renders the UI.*
- *Interaction: Communicates strictly with the backend API routes. For example, instead of querying the database directly, the Donor Assistant UI captures chat inputs and images, sending them over HTTP POST to the backend endpoints (/api/donation-assistant/*).*

2. Backend & Services (Next.js Serverless API Routes & Lib Services)

- *Role: Acts as the central orchestrator, API Gateway, and business logic execution layer.*
- *Interaction with Database: Ensures data integrity. For instance, when a donor confirms a donation, the backend communicates with Supabase (PostgreSQL) to create a donations row, insert donation_allocations, and append donation_events.*
- *Service Layer Isolation: Reusable logic sits in the src/lib/ directory. The Live Donation Matching Service (liveDonationMatching.ts) fetches 80 active needs from Supabase, scores them deterministically (using text overlaps, condition penalties, and disaster severity flags), and passes only the top candidates to the LLM.*

3. Database (Supabase PostgreSQL)

- *Role: The single source of truth for the entire platform.*
- *Interaction: It guarantees persistence for organizations, needs, disaster_events, and platform_status. State changes here (e.g., an admin changing platform status to crisis) immediately cascade to the frontend (e.g., UI displays disaster badges) and dictate the behavior of the backend queries (e.g., active disaster needs are sorted to the top of donor browse results).*

4. External APIs (Z.AI, Twilio, Google Maps, News Feeds)

- *Role: Specialized capabilities that the backend delegates tasks to (LLM reasoning, SMS/WhatsApp delivery, routing math, real-world data).*
- *Interaction: The backend formats structured prompts and payloads for these external providers.*
- *AI Interaction Example: The backend sends a carefully curated JSON chunk of live Supabase needs to the Z.AI GLM API, requesting a JSON-only response allocating percentages. Once GLM replies, the backend validates that GLM didn't hallucinate (checking if the returned needId actually exists in the DB) before passing the final plan to the frontend.*
- *Logistics Interaction Example: The route planning job pulls active platform-delivery allocations from Supabase, packages the origin/destination*

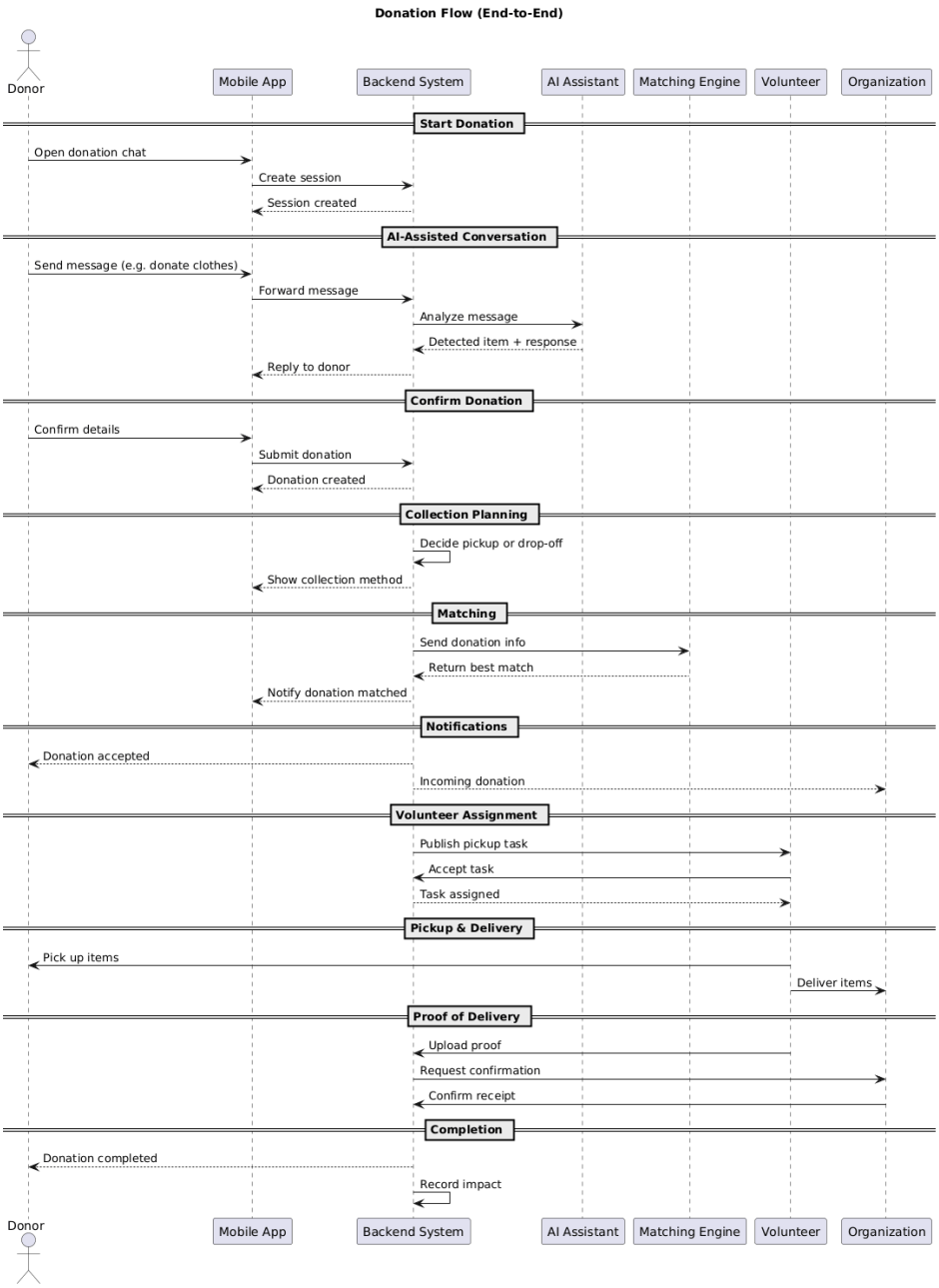
© UMHackathon 2026

coordinates, and calls Google Maps API to figure out optimal travel distances, updating the route_stops table with the resulting sequence.

g. *Must highlights the distributed components if the system is planning to be deployed across multiple services*
If this system is scaled across multiple services (e.g., AWS or GCP), it is already designed with distributed boundaries:
   - *Client Tier (Edge): The Next.js frontend (Client components) deployed globally on a CDN (like Vercel Edge or AWS CloudFront) handling UI state and capturing donor intentions.*
   - *API Gateway / Serverless Tier: The Next.js API routes act as the backend-for-frontend (BFF) and API Gateway. They handle authentication, orchestrate LLM calls, and validate payloads. They are stateless and horizontally scalable.*
   - *Asynchronous Worker Tier: The /api/jobs/* routes are designed to be decoupled. In a larger deployment, these would be separate worker microservices (e.g., AWS SQS + Lambda) handling heavy tasks like Route Planning, News Ingestion, and Equity Recomputing without blocking user-facing APIs.*
   - *State Management (Database): Supabase (PostgreSQL) acts as the single source of truth. All cross-component communication (e.g., a donor making a donation - receiver getting a notification) relies on database state changes (donation_events) rather than direct synchronous service-to-service calls, ensuring high resilience.*

© UMHackathon 2026

2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend

a. *Next.js (App Router): Core framework providing server-side rendering, routing, and optimized performance.*

b. *React & TypeScript: UI library and strongly typed language for building robust, modular components.*

c. *Tailwind CSS: Utility-first CSS framework for rapid, responsive styling.*

© UMHackathon 2026

2.2.2. Backend [e.g. FastAPI – RESTful API for the microservices]

a. Next.js Server Components / API Routes: Handles backend logic, session verification, and orchestration without needing a separate backend server.

b. Z.AI (GLM): integrated via API routes for automated donation item screening, condition analysis, and workflow orchestration.

c. Twilio: Enables communication services such as SMS or WhatsApp notifications for donors, volunteers, and organizations.

d. Brevo: Handles transactional emails such as donation confirmations, alerts, and system notifications.

e. Ngrok: Provides secure tunneling to expose local development servers for testing APIs, webhooks (e.g., Twilio), and external integrations.

2.2.3. Database [e.g. PostgreSQL – Relational Database for orders, users, vendors, and riders]

a. PostgreSQL (via Supabase): Relational database managing user profiles (donors/receivers), organizations, needs, corporate ledgers, and donation tracking.

2.2.4. Cloud/Deployment [e.g. AWS EC2 (Server), RDS (PostgreSQL), S3 (Static Assets)

a. Supabase Auth: Manages user authentication, sign-ups, and session tokens.

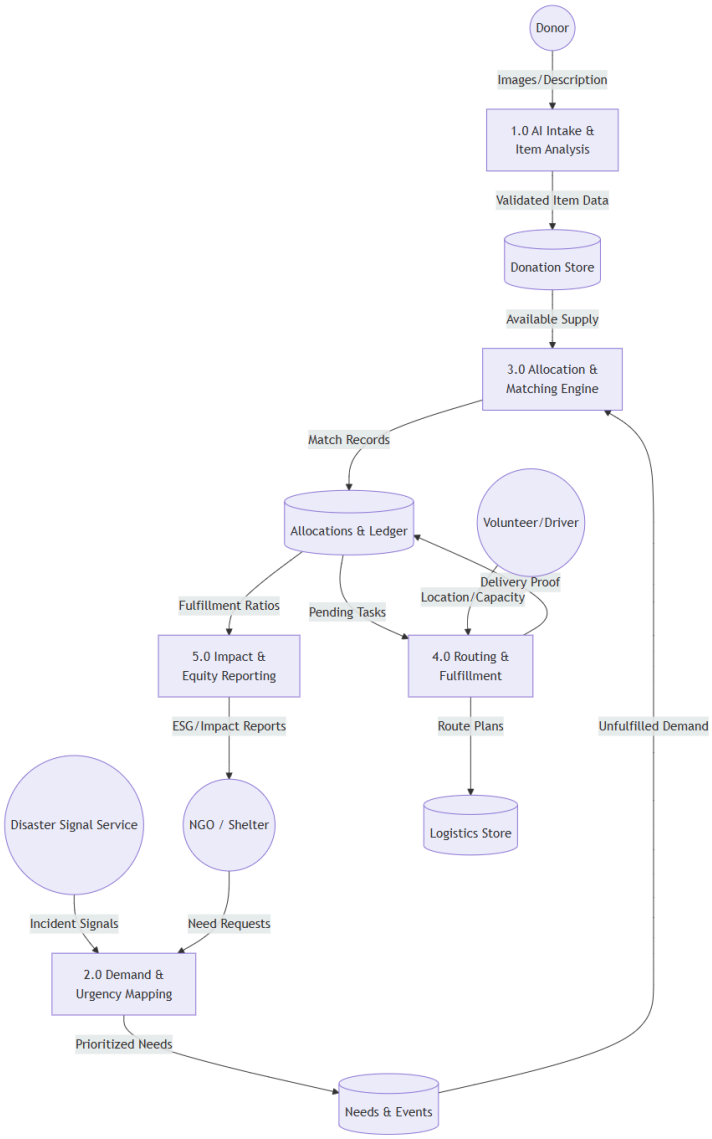
b. Supabase Storage: Cloud storage buckets (e.g., corporate_bulk_donation_photos) for hosting static assets and user-uploaded donation images.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

© UMHackathon 2026



2.3.2. **Normalized Database Schema**

Link access to ERD:
https://drive.google.com/file/d/1vV_4rQ0yQ9ySmsRTKUXKEHP8qK7rnyMX/view?usp=sharing

3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. ***[e.g. in Table]***

© UMHackathon 2026

#	Features	Description
1	Public Needs Browsing	Users can browse active needs posted by organizations in need without logging in, with basic filtering by category and urgency to quickly identify where help is required.
2	AI Donation Assistant	Donors interact with a guided AI chatbot to submit donation details, where the system processes input and recommends suitable organizations based on real-time needs.
3	Organization in Need Management	Organizations in need can create, update, and manage their needs, allowing them to communicate demand dynamically and receive matched donations.
4	Donation Confirmation & Tracking	After confirming a donation, the system records the transaction, creates allocation records, and allows donors to track the delivery progress through defined stages.
5	Admin Management	Admins can create and activate disaster events, manage incoming signals, and coordinate basic response workflows including need publishing and monitoring.

3.2. Non-Functional Requirements (NFRs)

The system qualities that are not features but are crucial for reliability and improve the system:

Quality	Requirements	Implementation
Scalability	The system must handle severe, unpredictable traffic spikes (from donors and receivers) simultaneously when a new disaster event is declared, without crashing the core web server.	Serverless Compute: Next.js Serverless API routes scale horizontally automatically (e.g., via Vercel). Decoupled Jobs: Heavy processes like route optimization and signal ingestion are offloaded to async endpoints (/api/jobs/*) triggered via cron, keeping user-facing APIs fast.
Reliability	The donation intake process must remain 90% available. If external LLMs or the live	Multi-Tier Fallbacks:

© UMHackathon 2026

	database go down, the donor must still be able to complete a donation flow.	*1. If Z.AI GLM fails, the system falls back to live DB deterministic heuristic matching.* *2. If Supabase DB matching fails, it falls back to a static offline catalog*
Maintainability	*The disaster-response features must not break the existing day-to-day generic donation flows. External third-party integrations should be easy to swap if a provider changes.*	*Modular Architecture: Migrated to Next.js App Router for strict domain boundaries.* *Provider Interfaces: Integrations are abstracted via dedicated boundary files (e.g., newsSources.ts, maps.ts, whatsapp.ts), meaning changing from Twilio to Meta API only requires touching one file.* *Schema Extension: Current models (organizations, needs) are extended safely rather than building a redundant parallel database.*
Cost Efficiency	*LLM token usage and database read costs must be strictly bounded to prevent budget overruns during mass-donation events.*	*The system prevents context bloat by reading a maximum of 80 active needs from Supabase. It deterministically scores them locally, sending only a few top candidates to the GLM API. Prompts are strictly limited (max_tokens: 500) and enforce JSON-only outputs.*

3.3. Out of Scope / Future Enhancements

The following features are part of the DonateAI grand vision but are not included in the UMHackathon 2026:

a. *Native mobile application wrappers for iOS and Android*
b. *Live GPS tracking of third-party dispatch riders within the DonateAI interface*
c. *In-app chat between donors and NGO staff after matching*
d. *Scheduled or time-delayed donation pickups*

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

To ensure the stability of critical disaster-response workflows (such as AI-assisted need extraction and intelligent route planning), our deployment strategy relies heavily on controlled, phased releases.

- *Grayscale Rollout: When introducing significant updates like a new version of the GLM reasoning engine for live donation matching, the feature is initially released to 5% of real user traffic (e.g., donors in a specific, low-risk geographical zone). System health metrics such as AI latency, hallucination rates, and Supabase query performance are closely monitored. If no anomalies are detected within the initial observation window, traffic is gradually ramped up to 20%, 50%, and finally 100%.*
- *A/B Testing: Once a feature passes the grayscale rollout, we utilize A/B testing to optimize operational outcomes. For example, we might route 50% of automated volunteer delivery assignments using an algorithm optimized strictly for shortest travel distance (Variant A), and 50% using an algorithm balanced for shelter equity and underserved scoring (Variant B). We then evaluate which variant yields higher overall shelter fulfillment rates without causing volunteer burnout, establishing the winning variant as the new baseline.*

4.1.2. Emergency Rollback (ER) & Golden Release:

Given the mission-critical nature of disaster operations, system resilience and rapid recovery are our top priorities.

- *Traffic Management & Monitoring: We continuously monitor core pathways, such as the Twilio WhatsApp webhook ingestion and the Z.AI Vision screening endpoints. If the system detects severe anomalies—such as image screening API timeouts exceeding our defined threshold or database read failures—an automated Emergency Rollback (ER) is instantly triggered.*
- *Golden Release Deployment: Upon triggering ER, the CI/CD pipeline automatically shifts traffic back to the Golden Release (the last known stable, certified version of the platform). In the context of our donation matching logic, if the live GLM Supabase matching service fails, the Golden Release includes a failover mechanism that safely routes donor traffic to the deterministic, static NGO demand catalog. This ensures donors can still pledge items and shelters can still receive baseline help while the engineering team investigates the disruption.*

4.1.3. Priority Matrix:

© UMHackathon 2026

4.1.3.1.

P1: Critical	*System outage or core workflow completely broken; halts disaster operations.*	*AI donation image screening failure rate > 1%- WhatsApp shelter outreach webhook completely unresponsive- Next.js platform or Supabase database is down*	***Immediate ER to Golden Release.*** *Failover to static fallback catalogs. On-call engineers paged immediately. Resolution expected within < 1 hour.*
P2: High	*Major feature degraded, but core system functions or a workaround exists.*	*GLM route optimization taking > 30 seconds- Google Maps API rate limits exceeded- Delay in automated Facebook/IG campaign posts*	*Toggle affected subsystem to "Manual Override" (e.g., admins manually assign routes). Fix deployed within 2–4 hours.*
P3: Medium	*Non-critical functionality impaired; user experience degraded but operations continue.*	*Equity monitoring dashboard charts failing to load- Minor discrepancies in post-crisis report summaries- Delayed SMS/Email fallback notifications*	*Logged as a high-priority bug. To be fixed in the next scheduled sprint or patch release (1–3 days).*
P4: Low	*Minor visual bugs, typos, or localized UI glitches.*	*Misaligned buttons on the receiver portal- Typos in the Bahasa Melayu localization- Chatbot UI animation stutter*	*Logged in the general backlog for resolution during regular maintenance cycles.*

4.2. Monitoring:

4.2.1. The following metrics will be tracked in real-time via an application monitoring dashboard:

- *LLM API Response Time (p50, p95, p99) — alert if p95 > 3 seconds*
- *JSON Extraction Accuracy Rate — (sessions with zero missing-field loops / total sessions) — alert if < 80%*
- *NGO Match Rate — (intakes resulting in a confirmed match / total intakes) — baseline target: 70%*
- *Logistics Wallet Balance — alert if balance drops below minimum threshold (e.g., RM 500)*
- *Supabase Query Latency — alert if p95 > 500ms*
- *Email Delivery Success Rate — alert if < 90% via Brevo dashboard*

© UMHackathon 2026

- *System Uptime — 99% target for the intake-to-match flow*

4.3. Assumptions:

The following operational and environmental assumptions are considered valid during DonateAI development:

- *Donors have stable internet connections on their devices during the intake chat session.*
- *NGOs will proactively maintain and update their needs catalog entries (mark as fulfilled when satisfied).*
- *Corporate ESG sponsors pre-load the Logistics Wallet with sufficient balance before the demo.*

4.4. External Dependencies

Tools	Purpose	Risks
Z.AI ILMU-GLM-5.1 API	Core LLM reasoning engine — handles prompt inference, multi-step agentic conversation, and structured JSON extraction	HIGH — if GLM API rate-limits or goes down, core intake flow fails
Supabase PostgreSQL	NGO needs catalog, donor intake storage, logistics order tracking, corporate wallet ledger	MEDIUM — free-tier row limits; risk of connection pooling exhaustion under surge load
Brevo SMTP	OTP verification emails and transactional routing confirmation emails	LOW — email delivery is non-blocking for core matching flow

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline: *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

	Day1	Day2	Day3	Day4	Day5	Day6	Day7	Day8	Day9	Day10
Project Planning										
UI/UX										
Documentation										

© UMHackathon 2026

Backend Features Coding										
AI Integration										
Pitching Video										
Submission										

5.2. Team Members Role:

Member	*Role*
Thian Xin Yi (Leader)	*Full-Stack Developer, focus on public view and donor view*
Chong Chee Yan	*Full-Stack Developer, focus on donor view*
Cheng Jia Ying	*Full-Stack Developer, focus on receiver and admin view*
Goh Qian Ying	*Full-Stack Developer, focus on receiver and corporate view*

© UMHackathon 2026